ONG CONNECT

Relatório acadêmico

Modelagem híbrida relacional e documental com JSON no MariaDB

Modelagem de Banco de Dados • Revisão: 27/10/2026

Integrantes do grupo

Nome	RGM
Daniel Costa Ferreira	048098108
Daniel Paulino	048711152
Dioni Alves Moreira	047399112
Igor Roehr lúcio de Godoy Strauss	5947848251
Luiz Carlos Domeneguetti Lourenção	047147695
Luiz Paulo Gomes do Prado e Silva	048424170
Raphael Vital	047147768

Instituição: Cruzeiro do Sul — Campus Paulista. **Turma:** Modelagem de Bancos de Dados — terça-feira, noite.

Apresentação prevista: 27/10/2026.

Sumário

ONG CONNECT 1

Relatório acadêmico 1

1. Estudo acadêmico 3

2. Modelo de dados e relacionamentos 7

3. Experimento e resultados 16

4. Referências 18

1. Estudo acadêmico

Resumo

O trabalho estuda o uso de documentos JSON junto a entidades relacionais no MariaDB, utilizando um portal de organizações sociais como caso prático. O domínio contém contas, ONGs, projetos, oportunidades, candidaturas e recebimentos simulados. Os identificadores, vínculos, estados e valores financeiros são modelados em colunas e relações; alguns atributos variáveis e listas ficam em documentos JSON. A comparação usa uma representação tradicional com uma tabela de dias por oportunidade e uma representação documental com a lista no JSON. Um experimento local reproduzível compara equivalência das consultas, extensão do cadastro e validação sintática. O resultado sustenta a flexibilidade do documento para esse exemplo, sem demonstrar superioridade geral de desempenho. O DER e os scripts permitem relacionar a pesquisa à implementação real.

Palavras-chave: modelagem de dados; MariaDB; JSON; integridade referencial; dados semiestruturados.

1. Problema e objetivo

Oportunidades de voluntariado podem possuir modalidade, local, vários dias de atuação, horários e orientações distintas. A identidade da oportunidade e sua ligação ao projeto são estáveis; parte do cadastro pode evoluir. Como representar essa variedade sem perder os vínculos com projetos e candidaturas?

O objetivo geral é avaliar uma modelagem híbrida de dados relacionais e documentos JSON no contexto do ONG Connect. Os objetivos específicos são descrever fundamentos, explicitar benefícios e custos, apresentar o modelo relacional real, construir um exemplo comparável e interpretar os resultados com limitações.

2. Tecnologia escolhida e delimitação do enquadramento

O recorte proposto é **gestão híbrida relacional/documental com SQL e JSON em um mesmo SGBD**, associado à pesquisa de gestão multimodelo. A literatura sobre bancos multimodelo discute a convivência de modelos para atender à variedade de dados [1]. O projeto demonstra um recorte limitado dessa ideia, não uma implementação própria de SGBD multimodelo nem uma comparação com bancos de grafos ou vetoriais.

MariaDB e JSON não são tecnologias recém-lançadas. O recorte estuda a combinação de modelos relacionais e documentais; seu enquadramento como tecnologia emergente na disciplina permanece sujeito à avaliação do professor.

3. Fundamentação bibliográfica e técnica

3.1 Variedade de dados e composição de modelos

Lu e Holubová [1] apresentam a gestão multimodelo como uma linha de investigação sobre diferentes representações em uma plataforma de dados. Este estudo usa essa discussão como motivação para combinar um núcleo de vínculos relacionais com alguns documentos, sem tratar todos os tipos de dado como intercambiáveis.

3.2 Documentos e validação

JSON é um formato textual para intercâmbio de dados, conforme a RFC 8259 [2]. No caso estudado, objetos descrevem atributos e arrays representam listas. Isso permite guardar informações adicionais sem acrescentar imediatamente uma coluna à tabela; não elimina a necessidade de definir estrutura, significado e políticas de validação.

O MariaDB documenta seu alias `JSON` como `LONGTEXT`, com validação por `JSON_VALID` [3,4]. A implementação do portal usa explicitamente texto e `CHECK`. Validar sintaxe não valida a regra de negócio: por exemplo, `{"vagas":"muitas"}` pode ser JSON sintaticamente correto e continuar inadequado ao cadastro. A API valida campos, tipos e agenda.

3.3 Índices e portabilidade

Colunas geradas podem materializar ou derivar valores e suportar índices no MariaDB [5]. O portal usa uma coluna gerada para a unicidade de candidaturas ativas; isso não significa que seus atributos JSON já tenham índices derivados. Tal alternativa não foi implementada como parte deste estudo.

PostgreSQL distingue `json` textual e `jsonb` com outra representação e recursos de indexação [6]. Essa diferença serve como alerta de portabilidade: os resultados deste experimento em MariaDB não são resultados de `jsonb` ou de outro SGBD.

As referências completas estão no capítulo 4 deste documento. As afirmações sobre o funcionamento do portal são verificadas no schema, migrações e API, não deduzidas apenas das fontes externas.

4. Domínio e aplicação prática

O portal possui três perfis autenticados: administrador geral, responsável por ONG e voluntário. A associação de responsáveis e ONGs é N:N, por `usuarios_ongs`. A ONG organiza projetos; cada projeto oferece oportunidades. Uma candidatura referencia um usuário e uma oportunidade. Valores privados e públicos são separados e não são misturados como se fossem a mesma operação.

O DER apresentado no capítulo 2 identifica as tabelas, chaves e cardinalidades. A fonte física combina o schema inicial e sete migrações. O sistema real utiliza:

Núcleo relacional	Informação documental
Projeto, ONG e segmento por FK	Detalhes variáveis da oportunidade em `dados`
Usuário, oportunidade e status da candidatura	Dados da inscrição em `candidaturas.dados`
Conta, e-mail único e perfil	Informações complementares em `dados_voluntario`
Autor, recurso e ação de auditoria	Metadados permitidos em `detalhes`

Valores financeiros permanecem em `DECIMAL(12,2)`, e não em campos JSON arbitrários. Os arquivos de documentos ficam em armazenamento privado; o banco guarda metadados e vínculos. Preferências locais não substituem os cadastros compartilhados.

Exemplo real de documento

```
{
  "titulo": "Apoio nas oficinas",
  "modalidade": "Presencial",
  "local": "Sede fictícia",
  "vagas": 5,
  "diasAtuacao": ["SEG", "QUA"],
  "horarioInicio": "09:00",
  "horarioFim": "11:00",
  "ativa": true
}
```

O documento não contém a única referência do projeto: `oportunidades.projeto_id` é uma FK real. A validação de capacidade/agenda ocorre em transações nas operações relevantes. `JSON_VALID` não deve receber crédito por garantias que são implementadas no PHP ou pelos índices.

5. Comparação com abordagem tradicional

O comparador relacional possui uma linha de oportunidade com atributos estáveis e uma tabela associativa `dias`, com uma linha para cada dia. O comparador documental armazena uma linha por oportunidade e um array `diasAtuacao`. Ambos representam os mesmos 2.000 registros sintéticos e retornam 1.000 oportunidades de segunda-feira.

Critério	Colunas e associação relacional	Documento JSON com IDs relacionais
Lista de dias	Linhas na associação; índice por dia	Array no documento; extração durante a consulta testada
Novo atributo no exemplo	Acrescentar coluna na tabela do experimento	Acrescentar chave no documento
Obrigatoriedade e tipos	Podem ser expressos nas colunas/restrições	Exigem validação além da sintaxe do JSON
Vínculos de negócio	FKs em colunas	FKs mantidas fora dos documentos no portal
Consulta frequente por um atributo	Índice explícito e plano previsível	Pode exigir normalização ou extração indexada
Melhor utilização neste caso	Identidade, estados, valores, relações e consultas frequentes	Complementos variáveis e listas de baixo volume

Essa comparação não implica que todo modelo relacional exija sempre `ALTER TABLE` para extensibilidade: ele também admite tabelas auxiliares ou outros desenhos. O experimento compara dois desenhos explícitos, não todas as alternativas possíveis.

6. Método experimental e resultados

O script `scripts/academic-json-comparison.php` cria apenas tabelas temporárias. Não altera cadastros do portal nem o schema permanente. São usados dados determinísticos fictícios, 20 repetições aquecidas por consulta, mesmo SGBD/conexão e verificação da equivalência dos resultados. O código testa também rejeição de JSON inválido e extensão de um atributo nas duas abordagens.

O capítulo 3 apresenta comandos, planos e valores efetivamente medidos. O resultado bruto fica em `resultados-experimento.json`. Não se apresentam números de produção ou uma medição em PostgreSQL.

O teste inicial mostrou retorno equivalente e mediana inferior para o desenho relacional indexado. No exemplo JSON, o novo atributo foi armazenado sem alterar a tabela, mas a busca na lista examinou os documentos. Assim, **a vantagem sustentada é de flexibilidade no exemplo, não de velocidade**. Essa conclusão deve ser mantida mesmo que o desempenho varie entre execuções.

7. Discussão crítica e ameaças à validade

A amostra é pequena, local, sintética e sem usuários concorrentes. A consulta relacional usa índice de dia; o comparador JSON não usa coluna gerada indexada. A diferença de planos faz parte da comparação, mas impede atribuir todo o resultado ao formato de dados. A ordem fixa das consultas, o cache aquecido, o hardware e a versão também limitam a generalização.

Não foram medidos escrita concorrente, custo de backup, tamanho em disco, replicação, JSON indexado ou resultados em outros SGBDs. As tabelas temporárias do microexperimento não possuem FKs entre si; as FKs da aplicação são evidenciadas no DER e nas migrações, e não atribuídas a esse microteste.

No sistema atual, dias e horários são lidos para verificar conflito de agenda. Caso consultas por esses campos cresçam, normalizar a agenda pode ser melhor do que ampliar documentos. A auditoria não preserva cada versão de cada campo, e relatórios por período consideram a situação atual dos registros. Esses limites não devem ser omitidos na demonstração.

8. Conclusão

O ONG Connect mostra que documentos JSON podem coexistir com relações explícitas e transações. O desenho preserva as garantias relacionais onde elas são essenciais e permite representar atributos complementares sem multiplicar imediatamente as colunas. O estudo também mostra o custo dessa flexibilidade: validação semântica adicional, consultas que precisam interpretar documentos e cuidado com indexação e portabilidade.

Para este domínio, a escolha recomendada é seletiva: manter identidades, recursos financeiros, permissões e vínculos em estruturas relacionais e avaliar JSON apenas onde a variedade justifica o custo. A demonstração, o DER e o experimento atendem aos componentes técnicos documentados.

2. Modelo de dados e relacionamentos

Visão geral das 18 tabelas

As linhas conectam as tabelas; as setas apontam para a tabela referenciada pela chave estrangeira. Os identificadores T01–T18 junto às FKs complementam a leitura. A autorreferência de SOLICITACOES_ONGS também está representada.



Todos os campos físicos estão representados. Os quadros têm altura proporcional ao conteúdo. A versão ampliada e as cardinalidades estão nas páginas seguintes; as tabelas técnicas não possuem vínculos de negócio.

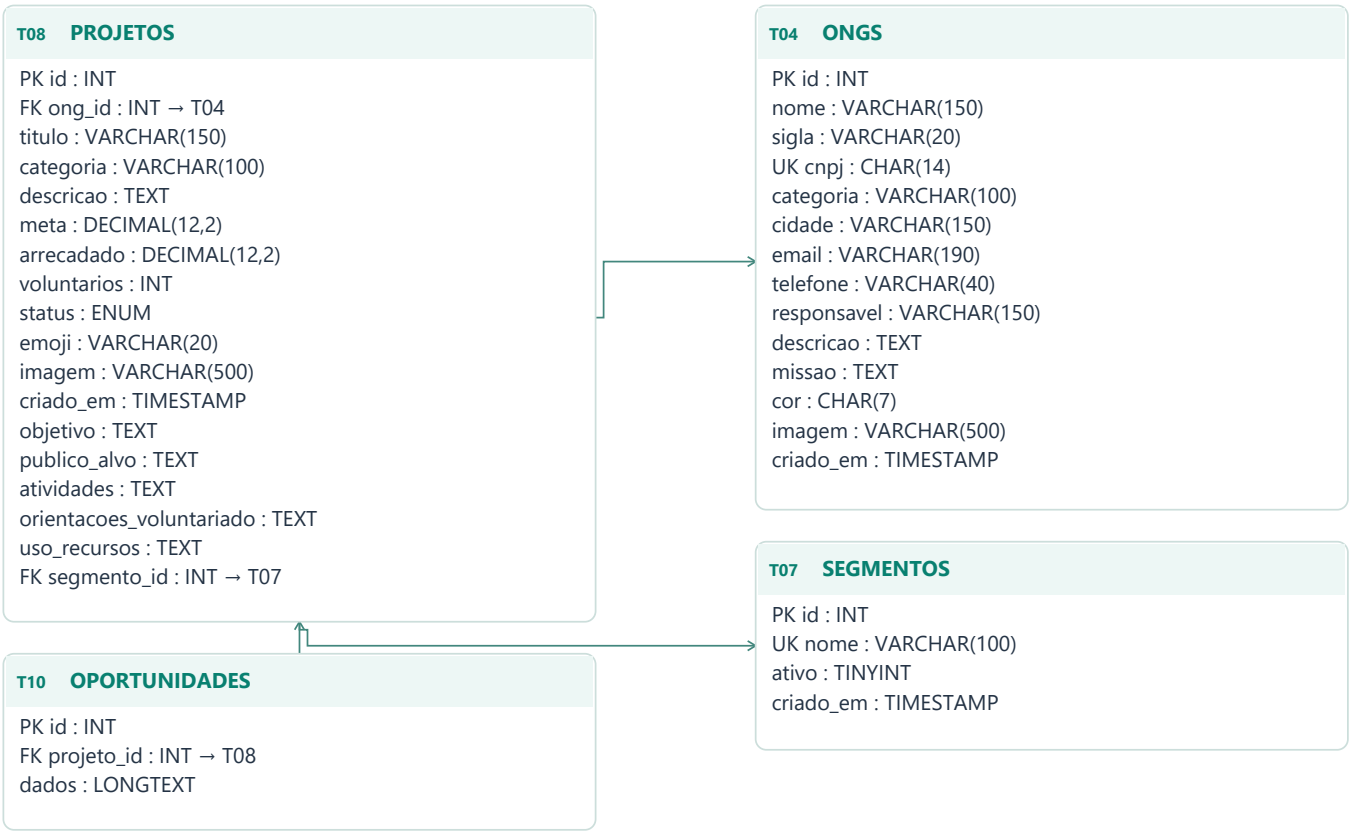
Revisado em **26/10/2026**. A referência física é `database/schema.sql` somado às migrações `001_shared.sql` a `007_segments.sql`, na pasta externa. Não execute apenas o schema inicial para representar o estado final.

Modelo conceitual e cardinalidades

Uma ONG possui projetos e responsáveis vinculados. Cada projeto seleciona um segmento e oferece oportunidades. Usuários voluntários se candidatam a oportunidades; cada candidatura possui decisões históricas. Doações privadas se relacionam a usuário e projeto. Repasses públicos pertencem a uma ONG e, opcionalmente, a um projeto da mesma ONG; podem ter documentos privados. Solicitações de ONG e seu histórico se vinculam ao controle permanente do CNPJ. A auditoria registra ações e pode manter o nome do autor mesmo após exclusão da conta.

Catálogo e classificação

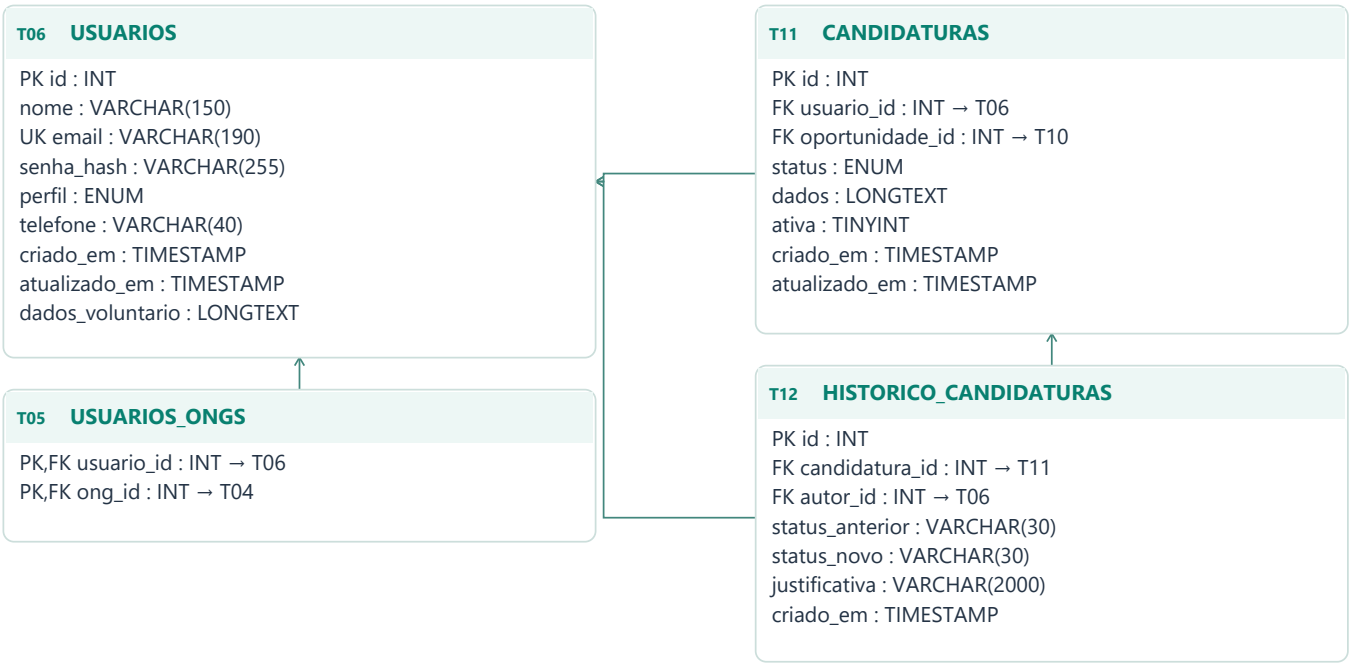
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: ONGS → PROJETOS: 1 : 0..N; SEGMENTOS → PROJETOS: 0..1 : 0..N; PROJETOS → OPORTUNIDADES: 1 : 0..N.

Contas e candidaturas

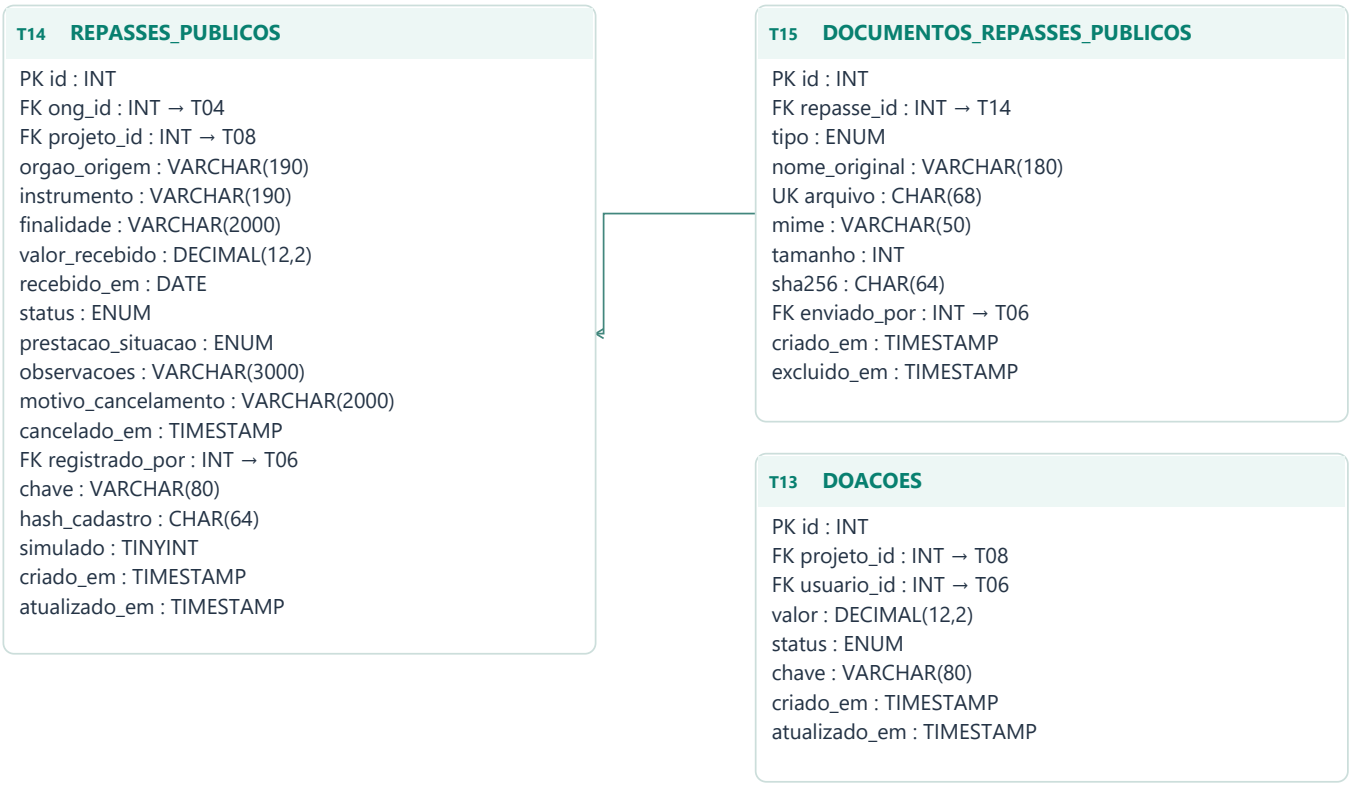
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: USUARIOS → USUARIOS_ONGS: 1 : 0..N; USUARIOS → CANDIDATURAS: 1 : 0..N; CANDIDATURAS → HISTORICO_CANDIDATURAS: 1 : 0..N; USUARIOS → HISTORICO_CANDIDATURAS: 1 : 0..N.

Doações, repasses e documentos

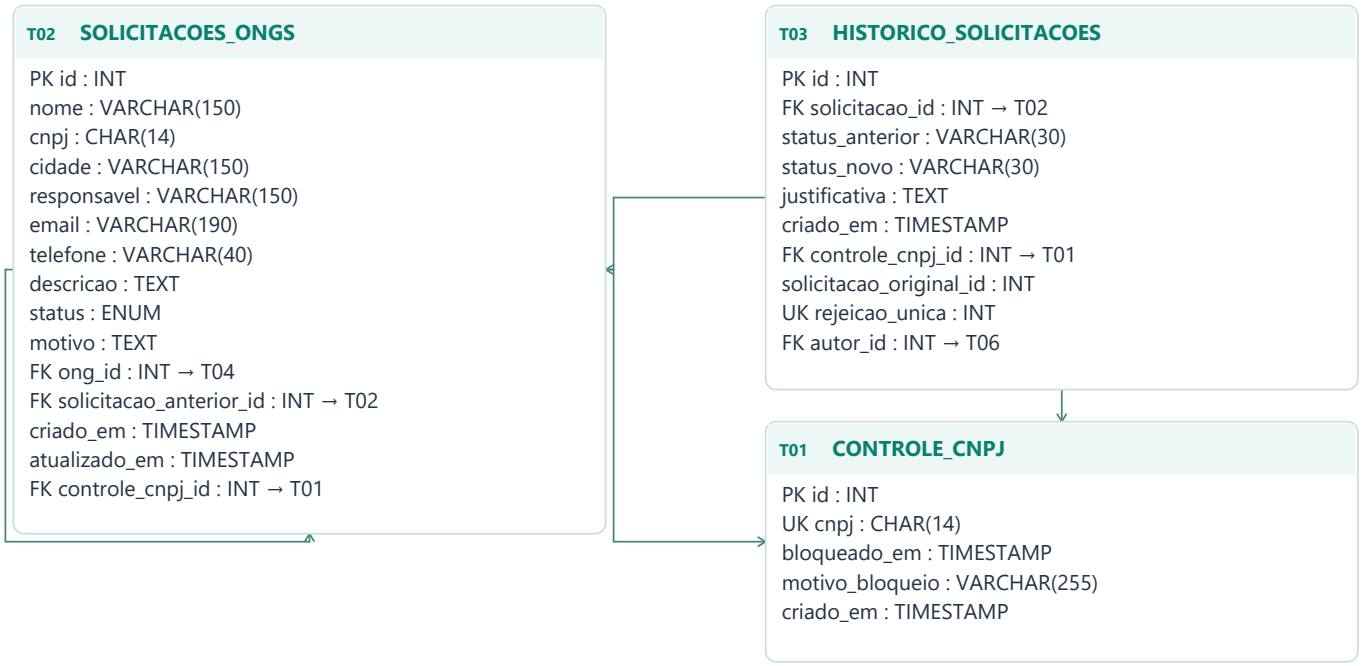
Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: REPASSES_PUBLICOS → DOCUMENTOS_REPASSES_PUBLICOS: 1 : 0..N.

Solicitações e controle de CNPJ

Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.



Cardinalidades: CONTROLE_CNPJ → SOLICITACOES_ONGS: 1 : 0..N; CONTROLE_CNPJ → HISTORICO_SOLICITACOES: 1 : 0..N; SOLICITACOES_ONGS → SOLICITACOES_ONGS: 0..1 : 0..N; SOLICITACOES_ONGS → HISTORICO_SOLICITACOES: 0..1 : 0..N.

Auditoria e tabelas técnicas

Todos os campos físicos, incluindo datas e atributos de apresentação. PK: chave primária; FK: estrangeira; UK: unicidade. Setas apontam à tabela referenciada. Referências Txx mantêm a identificação da visão geral, inclusive quando a tabela está em outra página.

T09 AUDITORIA

PK id : BIGINT
FK autor_id : INT → T06
recurso : VARCHAR(50)
registro_id : INT
acao : VARCHAR(30)
criado_em : TIMESTAMP
resultado : ENUM
detalhes : LONGTEXT
requisicao_id : CHAR(32)
autor_nome : VARCHAR(150)
autor_perfil : VARCHAR(20)

T16 LOGIN_TENTATIVAS

PK chave : CHAR(64)
tentativas : INT
inicio : DATETIME

T17 MVP_LOCK

PK id : INT
seeded : TINYINT

T18 SCHEMA_MIGRATIONS

PK versao : VARCHAR(100)
aplicado_em : TIMESTAMP

O diagrama representa os principais atributos e todas as tabelas, sem repetir cada campo de apresentação e timestamp. PK é chave primária, FK é estrangeira e UK indica unicidade. As três tabelas técnicas isoladas não têm relacionamentos de negócio. As cardinalidades opcionais representam a nulabilidade física; não eliminam exigências adicionais da API.

Dicionário e regras verificáveis

Tabela	Responsabilidade e restrições
`ongs`	Identificação e contato institucional; CNPJ único quando preenchido
`usuarios`	Conta, hash da senha, perfil e dados do voluntário; e-mail único
`usuarios_ongs`	Associação N:N de contas responsáveis e ONGs; PK composta evita vínculo repetido
`segmentos`	Vocabulário controlado; nome único, ativação/desativação sem apagar projetos
`projetos`	FK para ONG/segmento, descrições e meta; totais antigos `arrecadado`/`voluntarios` são legados, não a origem dos indicadores atuais
`oportunidades`	FK para projeto e JSON de título, vagas, modalidade, local, dias, horários e abertura
`candidaturas`	FK para usuário/oportunidade; índice único `(usuario_id,oportunidade_id,ativa)` impede duplicidade ativa e permite histórico inativo
`historico_candidaturas`	Transições com autor, justificativa e data; não substitui toda a fotografia de cada campo
`doacoes`	Valor positivo `DECIMAL(12,2)` e chave única por usuário; simulação idempotente
`repasses_publicos`	Recebimento positivo, origem, instrumento, prestação de contas e cancelamento; chave composta `(projeto_id,ong_id)` impede projeto de outra ONG
`documentos_repasses_publicos`	Metadados do arquivo, autor, integridade SHA-256 e exclusão lógica; conteúdo não fica no banco
`controle_cnpj`	Chave normalizada persistente, bloqueio após três rejeições distintas
`solicitacoes_ongs`	Pedido, situação, ligação com controle e eventual ONG publicada
`historico_solicitacoes`	Decisões preservadas após exclusão; rejeição contabilizada uma vez por solicitação
`auditoria`	Autor, alvo, ação, resultado e correlação; permite ações públicas sem autor autenticado
`login_tentativas`	Contador e janela temporal para limitar tentativas
`mvp_lock`	Serialização de determinadas gravações e controle da carga inicial
`schema_migrations`	Versões aplicadas pelo migrador

Integridade no banco e na API

FKs garantem existência dos vínculos. `DECIMAL` mantém representação decimal dos valores armazenados. Índices únicos protegem CNPJ, e-mail e reenvios. Transações e bloqueios impedem exceder a última vaga e preservam a sequência das decisões. A API valida perfis, dias/horários, capacidade, datas e escopo da ONG; essas regras não são garantidas apenas pelo DER.

A categoria textual de `projetos` é mantida por compatibilidade e atualizada junto ao segmento. O ID é a referência controlada. Uma alteração SQL direta que ignore a API pode desalinhar a cópia do nome; não se declara normalização completa de todos os campos. `dados` e `dados_voluntario` também são uma escolha de desnormalização parcial, analisada no trabalho.

Relacional + JSON efetivamente utilizado

Os documentos são guardados em `LONGTEXT` com `CHECK(JSON_VALID(...))`, e não em `jsonb`. Exemplos: `oportunidades.dados`, `candidaturas.dados`, `usuarios.dados_voluntario` e `auditoria.detalhes`. IDs de negócio permanecem em colunas relacionais. A validação de JSON não obriga um documento a ter um número positivo de vagas nem valida relações; essas verificações estão no PHP.

No modelo tradicional alternativo do experimento, dias ficam em uma tabela associativa por oportunidade. No modelo efetivo, a lista é parte do documento. Consulte experimento para a comparação reproduzível; ele não altera o schema do portal.

Exclusões, documentos e histórico

Solicitações removidas podem deixar seu histórico ligado ao controle permanente de CNPJ, com FK da solicitação nula. Doações/repasses cancelados permanecem registrados; cancelamento não é exclusão. Documentos usam `excluido_em`; os backups precisam abranger banco e armazenamento privado. A auditoria não é um log inviolável de todas as versões e não tem uma FK para cada possível `registro_id`.

Evolução física

Migração	Alteração
`001_shared`	Contas, vínculos, candidaturas, decisões, doações, auditoria e tentativas de login
`002_legacy_schedule`	Compatibilidade da agenda legada
`003_cnpj_audit`	Controle permanente de CNPJ, preservação de histórico e auditoria ampliada
`004_project_details`	Objetivo, público, atividades e orientações de projetos
`005_public_transfers`	Repasses públicos e metadados dos documentos privados
`006_volunteer_profile`	Documento JSON de perfil do voluntário
`007_segments`	Segmentos controlados e vínculo aos projetos

As preferências do navegador não integram o DER. SQL, migrações e testes são a evidência executável das restrições.

3. Experimento e resultados

Executado localmente em **26/10/2026**; data e fuso da execução constam no resultado bruto. Script: `scripts/academic-json-comparison.php`, na pasta externa.

Perguntas verificadas

1. As duas representações retornam o mesmo conjunto esperado de oportunidades por dia?

2. Um atributo adicional pode ser colocado no JSON sem acrescentar coluna?

3. O banco rejeita sintaxe JSON inválida quando existe `CHECK(JSON_VALID(...))`?

4. Como se comportam os planos e tempos das duas consultas específicas?

Ambiente e dados

• PHP **8.5.12**, MariaDB **11.4.5-MariaDB**, conexão local PDO; sem acesso ao banco remoto.

• 2.000 oportunidades fictícias em cada desenho, IDs de 1 a 2.000.

• IDs ímpares têm SEG/QUA e pares têm TER/QUI, resultando em 4.000 linhas de dias na alternativa normalizada.

• Mesmos títulos, capacidades e distribuição; cada consulta por SEG encontra 1.000 oportunidades.

• Uma consulta inicial de aquecimento, seguida de 20 repetições medidas por `hrtime`.

• Mediana das posições centrais ordenadas e p95 pelo método da posição 19 em 20 (nearest rank).

• Tabelas `TEMPORARY` com InnoDB e prefixo `academic_`, restritas à conexão. Não modificam schema nem registros permanentes; desaparecem ao fechar a conexão. O código não usa `DROP DATABASE` nem cargas no portal.

Desenhos comparados

```
-- Relacional: colunas e associação para os dias.
CREATE TEMPORARY TABLE academic_rel (
  id INT PRIMARY KEY, titulo VARCHAR(100), vagas INT NOT NULL
) ENGINE=InnoDB;
CREATE TEMPORARY TABLE academic_dias (
  oportunidade_id INT NOT NULL, dia CHAR(3) NOT NULL,
  PRIMARY KEY(oportunidade_id,dia), INDEX por_dia(dia,oportunidade_id)
) ENGINE=InnoDB;

-- Documental dentro do mesmo SGBD.
CREATE TEMPORARY TABLE academic_json (
  id INT PRIMARY KEY,
  dados LONGTEXT NOT NULL CHECK(JSON_VALID(dados))
) ENGINE=InnoDB;
```

As tabelas temporárias não implementam FKs entre si. A demonstração das FKs pertence ao modelo real do portal, que contém a relação oportunidade–projeto e os demais vínculos. Não se atribui ao microexperimento uma garantia que ele não testa.

```
SELECT COUNT(*)
FROM academic_rel o
JOIN academic_dias d ON d.oportunidade_id=o.id
WHERE d.dia='SEG';

SELECT COUNT(*)
FROM academic_json
WHERE JSON_CONTAINS(dados,'"SEG"','$.diasAtuacao');
```

Resultados da execução registrada

Medida	Relacional indexado	JSON sem índice derivado
Registros retornados na contagem	1000	1000
Mínimo por consulta	2.022 ms	3.205 ms
Mediana por consulta	2.138 ms	3.375 ms
p95 por consulta	2.68 ms	4.236 ms
Repetições medidas	20	20

O `EXPLAIN` foi gravado no JSON dos resultados. Na execução registrada, a alternativa relacional utilizou o índice `por_dia` e lookup pela PK; a alternativa documental realizou leitura integral para avaliar o array. A medição não é evidência de que todo uso de JSON é lento: é o resultado destes desenhos e desta consulta.

O teste também confirmou:

• JSON malformado rejeitado pelo banco: **sim**.

• Chave `acessibilidade` acrescentada via `JSON_SET`, sem `ALTER TABLE`: **sim**.

• Coluna `acessibilidade` acrescentada ao desenho relacional com `ALTER TABLE`, seguida de atualização: executado no ambiente temporário.

A facilidade de adicionar a chave foi observada funcionalmente, não como um benchmark de custo de migração. Outros desenhos relacionais também poderiam tratar extensibilidade de outras formas.

Interpretação e limites

O benefício observado do documento é a flexibilidade da representação complementar; a consulta relacional indexada foi mais rápida nesta execução. Não se generalizam os números para PostgreSQL, MySQL, produção, outro volume ou JSON com coluna gerada indexada.

A avaliação é local, aquecida, com uma distribuição artificial, 20 repetições e ordem fixa dos desenhos. Não mede rede, múltiplos usuários, escrita concorrente, custo em disco ou migração real de milhares de linhas permanentes. O índice na associação e a ausência de índice derivado no documento são diferenças explícitas. Cache e ruído do computador afetam as medições.

O resultado não prova que o portal deve ser refeito nem que foi implementado um novo SGBD. Ele fornece uma comparação pequena e reproduzível para discutir escolhas de modelagem.

4. Referências

Consultadas em **26/10/2026**. Fontes primárias; paráfrases próprias, sem reprodução integral dos textos. Os números correspondem às citações do relatório.

1. **LU, Jiaheng; HOLUBOVÁ, Irena.** *Multi-model Databases: A New Journey to Handle the Variety of Data*. ACM Computing Surveys, v. 52, n. 3, artigo 55, 2019. DOI: 10.1145/3323214. Registro e manuscrito dos autores na Universidade de Helsinki. Fundamenta o contexto de gestão de dados variados; não certifica a implementação do ONG Connect.

2. **BRAY, T. (ed.); IETF.** *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259, dezembro de 2017. Texto oficial. Referência para o formato JSON; sua data evidencia que JSON não é uma criação recente.

3. **MARIADB.** *JSON Data Type*. Documentação oficial. Página. Referência do armazenamento e da diferença em relação a outras implementações.

4. **MARIADB.** *JSON_VALID*. Documentação oficial. Página. Referência de validação sintática.

5. **MARIADB.** *Generated Columns*. Documentação oficial. Página. Referência para colunas geradas e alternativas de indexação.

6. **POSTGRESQL GLOBAL DEVELOPMENT GROUP.** *JSON Types*, documentação PostgreSQL 16, seção 8.14. Página da versão consultada. Utilizada somente para discutir diferenças de implementação e portabilidade; PostgreSQL não foi executado no experimento.

Fontes primárias do próprio projeto

• Enunciado fornecido: **Projeto da disciplina — Modelagem de Banco de Dados — Terças (1).pdf**, duas páginas. Define os componentes da atividade, a data, o limite de integrantes e o exemplo de loja. O arquivo permanece no local disponibilizado pelo usuário; não é redistribuído nem convertido em instrução para envio de e-mail.

• `database/schema.sql` e `database/migrations/001...007`: evidência do modelo físico.

• `ong-connect/api/`: evidência das regras efetivamente verificadas no backend.

• `scripts/academic-json-comparison.php` e resultado bruto: evidência do experimento.

• `scripts/test-shared.cjs` e suítes chamadas: evidência dos testes funcionais descritos, separados do experimento de modelagem.

O formato acima é uma referência identificável e verificável; não se declara conformidade formal com um padrão bibliográfico que não foi exigido no PDF. Datas e critérios de acesso devem ser revistos se a equipe atualizar o estudo.